

university of groningen

Performance Engineering: Lecture #16

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-21

Overview



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

Challenge: Expensive Metadata Operations



- **Performance challenge**: Metadata operations can be expensive
  - This is particular relevant for parallel file systems
- **Optimization strategy**: Reduce metadata operations
  - Reduce the number of created files (e.g. use container file formats like **HDF5**)
  - Avoid check frequent file status checks
  - In case of parallel I/O: Use a shared file for multiple MPI ranks
  - In case the number of metadata operations cannot be reduced: Use file systems that are optimized for metadata handling

Challenge: Low I/O Bandwidth



- **Performance challenge**: I/O bandwidth is low compared to hardware capabilities
  - Note: Due to the complexity of storage subsystems, determining attainable I/O bandwidth can be difficult
  - Consider using benchmarks like IOR to determine attainable performance
- **Optimization strategies**:
  - Improve I/O patterns
  - In case of data locality: Use I/O buffers
  - Hide latency by means of I/O buffers or asynchronous I/O
  - In case of parallel I/O: Use cooperative I/O mechanisms (e.g. by using **MPI I/O**)

Improving I/O Patterns

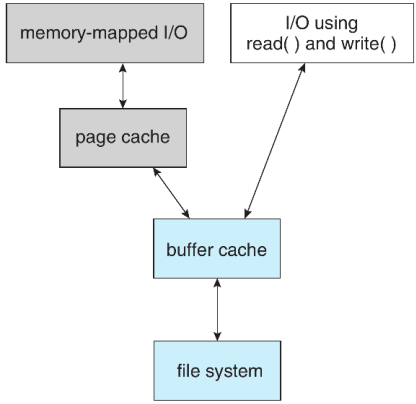


- **Sequential I/O**
  - Likely higher utilization of each block read from/written to storage
  - Benefit from lower-level optimization techniques like **read-ahead**
- **Large I/O records**
  - Records should be large with respect to file or storage hardware block size
  - Obtain required information: `stat -f <path>`
  - Example HAICGU:

```
1 [dpleiter@dev ~]$ stat -f /home/.
2   File: "/home/."
3     ID: 0         Namelen: 255     Type: nfs
4 Block size: 131072     Fundamental block size: 131072
5 Blocks: Total: 61022272   Free: 13962150   Available: 13962150
6 Inodes: Total: 781293760  Free: 769161482
```

Buffer Cache (1/2)

- **Strategy**: Maintain a buffer with copies of file system blocks in memory
  - Rationale:
    - Memory accesses are faster than storage device accesses
    - Much smaller penalties for small read-write accesses
- If a read or write request occurs the OS checks whether block is in the buffer cache
  - **Yes**: Execute I/O operation
  - **No**: Load block first before executing the I/O operation
- Need for a **replacement algorithm**
  - Typical choice: **Least Recently Used (LRU)**



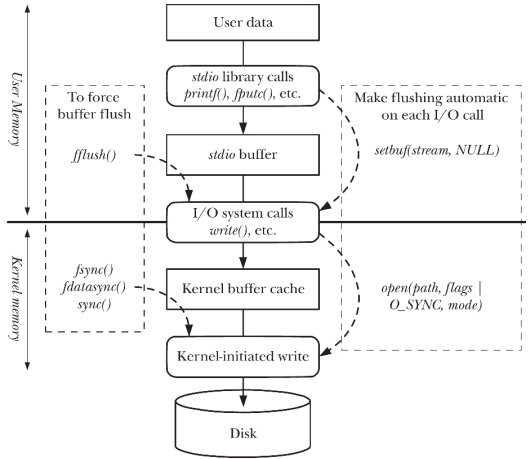
Buffer Cache (2/2)



- **Challenge**: The buffer cache and the virtual memory system compete for physical memory
  - This affects the choice of the buffer cache size
  - In case **buffered I/O** (usually the default) is not wanted, use **direct I/O**
    - Provide the `O_DIRECT` to `open()`
    - Databases often use direct I/O because they manage their own buffers
- **Optimizations** based on buffer caches: **Read-ahead**
  - If the OS detects a streaming read access to storage then it can trigger early read of blocks into the buffer cache
    - This applies to a case where the program reads from block $k, k+1, k+2, \ldots$
  - **Performance engineering opportunity**: Make sequential I/O easy detectable

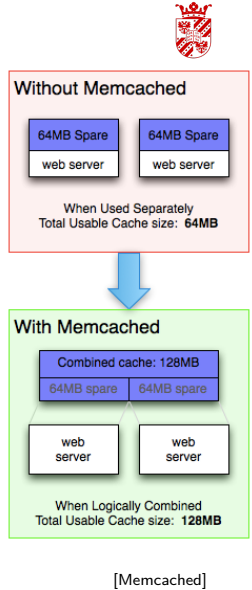
Linux I/O Buffering

- I/O can be buffered at **multiple levels**
  - Kernel level: buffer cache
  - **User level: POSIX stream buffers**
    - Used by functions like `fprintf()`, `fputs()`, `fscanf()`, `fgets()`
- Functions to manage POSIX stream buffers
  - `setvbuf()`: Assign buffering to a stream and set the type of buffering
  - `fflush()`: Flush a stream



[M. Kerrisk; 2010]

In-Memory Caches



- **Idea**: Keep frequently used data objects in an (distributed) in-memory cache
- **Example**: memcached [Brad Fitzpatrick; 2004]
  - In-memory key-value store with a distributed hash table

[Memcached]

Asynchronous I/O



- **Idea**:
  - Submit I/O read or write requests
  - Non-blocking function for submitting request
  - Blocking function to check to request completion
- **Example**: Linux-native asynchronous I/O facility `libaio`
  - Documentation: `https://pagure.io/libaio`

Question: Why is asynchronous I/O more important for read operations assuming the existence of a cache buffer?

Asynchronous I/O



- **Idea**:
  - Submit I/O read or write requests
  - Non-blocking function for submitting request
  - Blocking function to check to request completion
- **Example**: Linux-native asynchronous I/O facility `libaio`
  - Documentation: `https://pagure.io/libaio`

Question: Why is asynchronous I/O more important for read operations assuming the existence of a cache buffer?

Synchronous, blocking write operations typically return after data has been placed in the buffer cache. Synchronous, blocking read operations can likeley only complete once data arrived from disk.

libaio Example



```c
#include <libaio.h>
#include <stdio.h>
#include <fcntl.h>
#include <string.h>
#include <unistd.h>

int main() {
    io_context_t ctx = 0;                struct iocb cb;          struct iocb *cbs[1];
    struct io_event events[1];           int fd;                  int ret;
    struct timespec timeout = {0, 0};    char buf[128] = {0};

    io_setup(10, &ctx);                                  // 10 is the number of concurrent operations
    fd = open("slide.tex", O_RDONLY, 0644);

    io_prep_pread(&cb, fd, buf, 127, 0);                 // Initiate read
    cbs[0] = &cb;
    io_submit(ctx, 1, cbs);

    // DO SOMETHING ELSE

    io_getevents(ctx, 1, 1, events, &timeout);           // Wait for read to complete
    puts(buf);

    // Clean-up
    close(fd); io_destroy(ctx);

    return 0;
}
```

Content



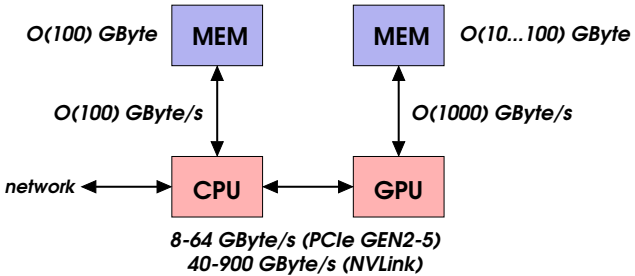
1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

GPU Computing



- GPU architecture is optimized for a **high degree of regular parallelism**
- **Relevant features**
  - High memory bandwidth
  - Highly multithreaded
  - Hardware thread scheduling
- GPUs enabled for HPC
  - NVIDIA: Hopper, Blackwell
  - AMD: Instinct MI250, Instinct MI300A
  - Intel: Ponte Vecchio
- GPUs are (typically) **not stand-alone devices** ☞ A **host CPU is required**

GPU Node Architecture



- Different memory devices
  - **Host memory** attached to CPU
  - **Device memory** attached to GPU
- Capacity and performance differ by $O(2 \dots 10)$

NVIDIA H100 Architecture

- NVIDIA Hopper architecture was introduced in 2022
- Up to 132 **Streaming Multiprocessors (SM)** with a shared L2 cache

- **Memory** (HBM3/HBM3e)
- **Host interface** (PCIe GEN5/NVLINK-C2C)
- **GPU-GPU interconnect** (NVLink)
- **Thread block scheduler** (GigaThread Engine)

[NVIDIA; 2022]



NVIDIA H100 SM

[NVIDIA; 2022]

- 4 processing blocks
- Processing per block
  - 16 FP64 units
  - 32 FP32 units
  - 1 mixed-precision Tensor Core
  - 8 load/store units
- Storage per SM
  - $4 \times 16,384$ 32-bit registers
  - 256 kByte shared memory/L1D cache
- Instructions
  - 1 shared L1I and 4 L0D caches
  - 4 warp schedulers
  - 4 dispatch units



NVIDIA H100 FP64/FP32 Performance



- IEEE 754-2008 double-/single-precision FMA
- Performance without tensor cores:
  - 2 Flop/cycle per floating-point unit
  - Double precision performance:
    $(132\,\text{SM}) \cdot (4 \cdot 16\,\text{FP64/SM}) \cdot (2\,\text{Flop/FP64/cycle}) = 16.896\,\text{Flop/cycle}$
    ☞ **Concurrency level required for high performance**
  - Assume $f = 1.98$ GHz (boost clock) $\Rightarrow$ $B_{\text{FP64}} = 33.5$ TFP64/s
  - FP32 operations throughput is $2\times$ larger

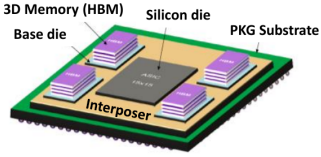
NVIDIA H100 Tensor Cores



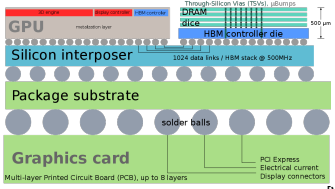
- Each Tensor Cores performs **matrix-multiply-accumulation**: $C \leftarrow A \times B + C$
- **Case of FP32**:
  - A, B, and C are 8×8 matrices
  - $B_{\mathrm{FP32,tensor}} = 8^3\,\mathrm{FP32/cycle/TC} = 270.336\,\mathrm{FP32/cycle/GPU}$
  - Assume $f = 1.83\,\mathrm{GHz}$ (boost clock) $\Rightarrow$ $B_{\mathrm{FP32}} = 494.7\,\mathrm{TFlop/s}$
- **Other data formats**
  - FP64
  - FP16, BFLOAT16
  - FP8 (different variants)
  - INT8

NVIDIA H100 Memory

- In-package HBM3 memory
- 5 memory stacks, $16\,\text{GByte/stack}$
  $\Rightarrow$ $C_{\text{mem}} = 80\,\text{GiB}$ **per GPU**
- A very wide bus of $5 \cdot 2 \cdot 512\,\text{bit}$ and a data rate of about $5\,\text{GT/s}$
  $\Rightarrow$ $B_{\text{mem}} = 3.2\,\text{TB/s}$ **per GPU**



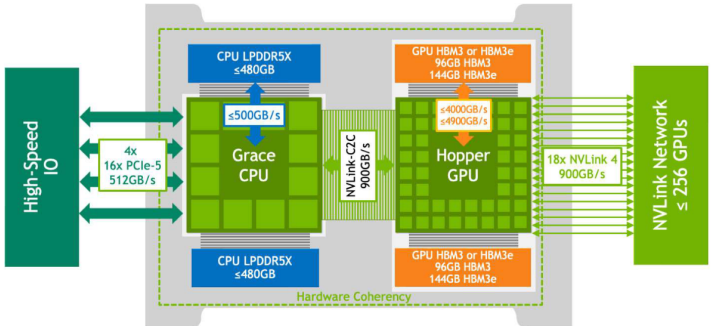
[H. Jun (SK Hynix), 2013]



[Wikipedia]

NVIDIA Grace-Hopper Superchip GH200

[NVIDIA; 2025]

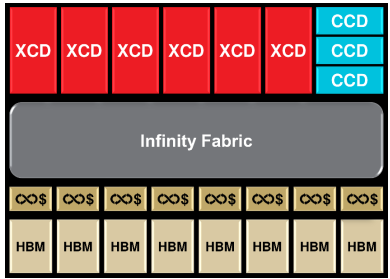


- NVIDIA started to integrate a CPU and GPU on a single daughterboard
- CPU-GPU Interconnect: NVLINK-C2C

AMD Instinct MI300a

- **Accelerated processing unit (APU) design**
  - CPU and GPU in a single package (multiple chiplets on an interposer)
  - Both access the same memory
  - $B_{\mathrm{FP64,vector}} = 61.3\,\mathrm{TFlop/s}$
  - $B_{\mathrm{FP64,matrix}} = 122.6\,\mathrm{TFlop/s}$
- $6\times$ accelerated compute dies (XCD)
- $3\times$ core complex dies (CCD)
  - $8\times$ Zen-4 cores per CCD
- $8\times$ HBM stacks
  - $B_{\mathrm{mem}} = 5.3\,\mathrm{TB/s}$
  - $C_{\mathrm{mem}} = 128\,\mathrm{GiB}$

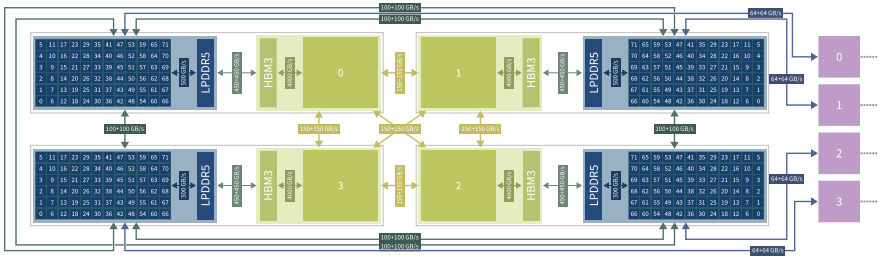
[J. Potyka et al. (AMD); 2025]



Multi-GPU Node

JUPITER node design with $4\times$ GH200 superchips:

[JSC; 2025]



- NVLink-C2C CPU-to-GPU links
- NVLink GPU-to-GPU links
- PCIe GEN5 links

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. **GPU Programming Models**
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

GPU Programming



- **Accelerator model**
  - Main application running on CPU
  - Kernel offload on GPU
  - Overall performance may suffer from Amdahl's law
- Typical kernel execution flow:
  - Allocate memory on GPU
  - Initialise memory on GPU or transfer data from host to GPU
  - Launch kernel
  - Transfer results from GPU to host
  - Deallocate memory on GPU

OpenMP



- As of version 4.0, the OpenMP API supports accelerators/coprocessors
- **Device model**
  - Host CPU (multi-threading)
  - One or more accelerator devices
- New construct: `target`

Example: $\vec{y} \leftarrow a \cdot \vec{x} + \vec{y}$

```
1  void saxpy() {
2    float a, x[SZ], y[SZ];
3    double t = 0.0;
4    double tb, te;
5
6    tb = omp_get_wtime();
7
8    #pragma omp target
9    for (int i = 0; i < SZ; i++) {
10     y[i] = a * x[i] + y[i];
11   }
12
13   te = omp_get_wtime();
14   t = te - tb;
15   printf("Time of kernel: %lf\n", t);
16 }
```

CUDA



- **CUDA** = Compute Unified Device Architecture
- C-like parallel programming language
- First SDK released in 2007
- Current version: 13.0.2 (October 2025)
- Language developed and controlled by NVIDIA
  - Only supported GPU devices are from NVIDIA
  - Available for x86, POWER, and Arm host CPUs

CUDA Kernel



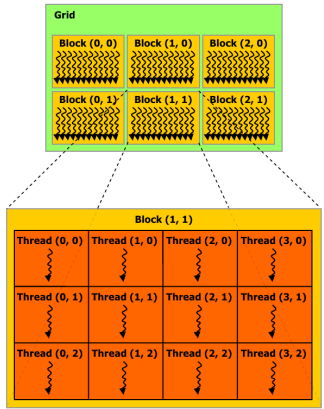
- **Kernel** = C function which is executed in parallel by $N \geq 1$ different threads
- Declaration specifiers:

`__global__`	Function on device, callable from host
`__device__`	Function on device, callable from device
`__host__`	Function on host, callable from host (default)

- Kernel launch syntax: `mykernel<<<Dg,Db>>>(...)`
  - `Dg`: Size of the grid
  - `Db`: Size of each block
- Each thread is (within a block) given a unique ID: `threadIdx`
  - `threadIdx` is structure with 3 components: `x`, `y`, `z`

CUDA Thread Hierarchy

- **Block** = Set of threads
  - 1-, 2- or 3-dimensional index
  - Synchronization within threads possible
  - Maximum number of threads: 1024
  - Branching with penalty
  - Identifier: `threadIdx`
- **Grid** = Set of blocks
  - 1-, 2- or 3-dimensional index
  - Synchronization not possible
  - Branching without penalty
  - Identifier: `blockIdx`



CUDA Vector Copy



```
1  #include <stdio.h>
2  #define N 32
3
4  __global__ void copy(float *b, float *a) {
5    int i = threadIdx.x;
6    b[i] = a[i];
7  }
8
9  int main() {
10   int i;
11   float ha[N], hb[N];                    /* Arrays on host */
12   float *da, *db;                        /* Pointer to arrays on device */
13
14   for (i = 0; i < N; i++) ha[i] = i;
15
16   cudaMalloc((void **) &da, N * sizeof(float));
17   cudaMalloc((void **) &db, N * sizeof(float));
18
19   cudaMemcpy(da, ha, N*sizeof(float), cudaMemcpyHostToDevice);
20
21   dim3 Dg(1,1,1); dim3 Db(N,1,1);
22   copy<<<Dg, Db>>>(db, da);
23
24   cudaMemcpy(hb, db, N*sizeof(float), cudaMemcpyDeviceToHost);
25
26   return 0;
27 }
```

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. **Device Utilization and Data Transfer**
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

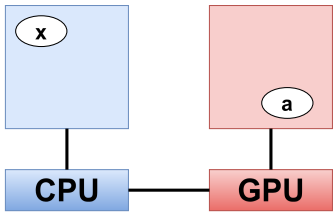
Overall Speed-up



- According to Amdahl's Law, the overall speed-up depends on
  - Speed-up factor S of the accelerated code parts ☞ Topic of the next section
  - Fraction f of the code that is accelerated
- Challenges for increasing f
  - A large fraction of the tasks run on the CPU only, while GPU is idle
  - Time spent on CPU↔GPU data transfer is large
- Performance engineering strategies
  - Port more tasks to the GPU
  - Overlap different tasks
  - Avoid unnecessary data transfer

Host versus Device Memory

- Device-accelerated nodes comprise of different memories
  - The address spaces are different, i.e. pointer can typically not be used for both memories
- Options for managing data
  1. Explicit data management by applications
  2. Implicit data management by software
  3. Implicit data management by hardware
- Implicit data management can be supported by hints



Overlapping Tasks



- **Consider the following sequence of tasks**:
  1. Start the application and initialise data structures on the CPU
  2. Transfer data from CPU to GPU memory
  3. Run a kernel on the GPU
  4. Transfer data from GPU to CPU memory
  5. Finish the application on the CPU
- **Latency in case of lacking overlap**:

$$\Delta t = \Delta t^{(1)}_{\text{CPU}} + \Delta t^{(2)}_{\text{CPU}\rightarrow\text{GPU}} + \Delta t^{(3)}_{\text{GPU}} + \Delta t^{(4)}_{\text{GPU}\rightarrow\text{CPU}} + \Delta t^{(5)}_{\text{CPU}}$$

- **Opportunities for overlapping tasks supported by the hardware**:
  - Tasks on CPU and GPU can execute independently
  - GPU DMA engines can transfer data independent of tasks running on CPU or GPU

Matrix-Vector Multiplication (1/3)



Example: $\vec{y} \leftarrow A \cdot \vec{x}$ (symmetric matrix of large size N)

Task	**Latency-bandwidth model**
Initialise A and x on CPU	$\Delta t^{(1)}_{\text{CPU}} = \lambda^{(1)} + (N^2 + N) \cdot w/\beta_{\text{mem,CPU}}$
Transfer A and x from CPU to GPU	$\Delta t^{(2)}_{\text{CPU}\to\text{GPU}} = \lambda^{(2)} + (N^2 + N) \cdot w/\beta_{\text{CPU}\leftrightarrow\text{GPU}}$
Compute $A \cdot \vec{x}$	$\Delta t^{(3)}_{\text{GPU}} = \lambda^{(3)} + (2 \cdot N^2 + N) \cdot w/\beta_{\text{mem,GPU}}$
Transfer y from GPU to CPU	$\Delta t^{(4)}_{\text{GPU}\to\text{CPU}} = \lambda^{(4)} + N \cdot w/\beta_{\text{CPU}\leftrightarrow\text{GPU}}$
Finish the application on the CPU	—

w ... size of the floating-point numbers

Matrix-Vector Multiplication (2/3)



Question: Is it a good idea to use the GPU in this case?

Matrix-Vector Multiplication (2/3)



Question: Is it a good idea to use the GPU in this case?

Most likely not!

- Typically, $\beta_{\text{mem,CPU}} \ll \beta_{\text{mem,GPU}}$
  $\Rightarrow$ Matrix-vector multiplication is expected to run faster on GPUs
- **But**: $\beta_{\text{CPU}\leftrightarrow\text{GPU}} \ll \beta_{\text{mem,GPU}}/2$
  $\Rightarrow$ Likely, most time will be spent on CPU$\leftrightarrow$GPU data transfer

Matrix-Vector Multiplication (3/3)



- Consider the **alternative example** where for $i = 0, 1, \ldots, M-1$ we need to compute

$$\vec{y}^{(i)} \leftarrow A \cdot \vec{x}^{(i)}$$

Question: Why is this example more suitable?

Matrix-Vector Multiplication (3/3)



- Consider the **alternative example** where for $i = 0, 1, \ldots, M-1$ we need to compute

$$\vec{y}^{(i)} \leftarrow A \cdot \vec{x}^{(i)}$$

Question: Why is this example more suitable?

- A should be copied only once
- For large M, the CPU↔GPU data transfer is dominated by transfer of $\vec{x}^{(i)}$ and $\vec{y}^{(i)}$
- Scaling according to the latency-bandwidth model

$$\begin{aligned} \Delta t^{(2,4)}_{\mathrm{CPU}\leftrightarrow\mathrm{GPU}} &\propto M \cdot 2 \cdot N \cdot w/\beta_{\mathrm{CPU}\leftrightarrow\mathrm{GPU}} \\ \Delta t^{(3)}_{\mathrm{GPU}} &\propto M \cdot (2 \cdot N^2 + N) \cdot w/\beta_{\mathrm{mem,GPU}} \end{aligned}$$

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. Final

Introduction



- Optimization targets considered in this section:
  - Instruction-throughput-limited applications: **maximise SM utilization**
    - Example: **Occupancy** = Ratio of active warps on an SM to the maximum number of active warps supported by the SM
  - Memory-performance-limited applications: **maximize memory interface utilization**

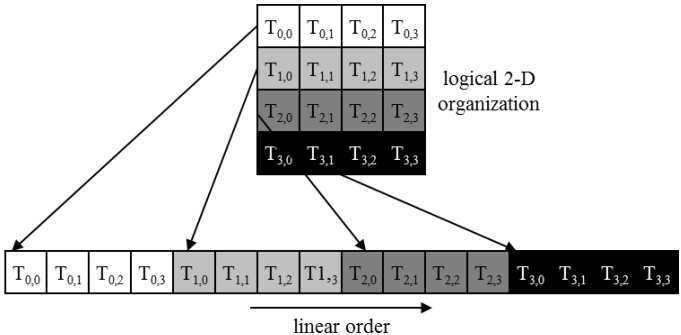
Warps as Scheduling Units



- Each block is divided into **thread warps** running multiple threads
  - An implementation technique (not part of the CUDA programming model)
  - Warps are scheduling units in SM
  - Threads in a warp execute in Single Instruction Multiple Data (SIMD) manner
  - The number of threads in a warp may vary in future generations

Warps in Multi-dimensional Thread Blocks

- The thread blocks are first linearized into 1D in row major order
  - In x-dimension first, y-dimension next, and z-dimension last



Blocks are Partitioned after Linearization



- Linearized thread blocks are partitioned
  - Thread indices within a warp are consecutive and increasing
  - Warp 0 starts with Thread 0
- Partitioning scheme is consistent across devices
  - Thus you can use this knowledge in control flow
  - However, the exact size of warps may change from generation to generation
- DO NOT rely on any ordering within or between warps
  - If there are any dependencies between threads, you must `__syncthreads()` to get correct results

SMs are SIMD Processors



- SM acts as control unit for instruction fetch, decode, and control for multiple shared processing units
- All threads in a warp must **execute the same instruction at any point in time**
- This works efficiently if all threads follow the same control flow path
  - All if-then-else statements make the same decision
  - All loops iterate the same number of times

Control Divergence



- Control divergence occurs when threads in a warp take different control flow paths by making different control decisions
  - Some take the then-path and others take the else-path of an if-statement
  - Some threads take different number of loop iterations than others
- The execution of threads taking different paths are serialized in current GPUs
  - The control paths taken by the threads in a warp are traversed one at a time until there is no more
  - During the execution of each path, all threads taking that path will be executed in parallel
  - The number of different paths can be large when considering nested control flow statements

Control Divergence Examples



- Divergence can arise when branch or loop condition is a function of thread indices
- Example kernel statement with divergence:

```
1 if (threadIdx.x > 2)
2 { ... }
```

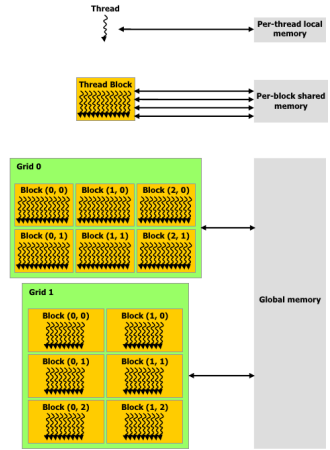
  - This creates two different control paths for threads in a block
  - Decision granularity $<$ warp size
  - Threads 0, 1 and 2 follow different path than the rest of the threads in the first warp
- Example without divergence:

```
1 if (blockIdx.x > 2)
2 { ... }
```

  - Decision granularity is a multiple of blocks size
  - All threads in any given warp follow the same path

CUDA Memory Hierarchy

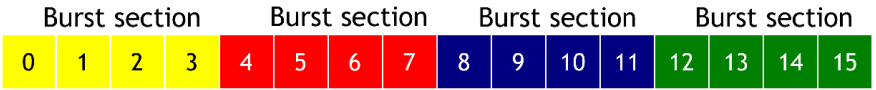
- Each thread can access registers and local memory
- Shared memory is visible to all threads of a block
- **Global memory space**
  - Resides in device memory
  - Accessible by all threads
- **Texture memory space**
  - Resides in device memory
  - Read via texture cache optimized for 2-d locality
- **Constant memory space**
  - Resides in device memory
  - Read-only, not dependent on thread ID



Memory Coalescing (1/2)

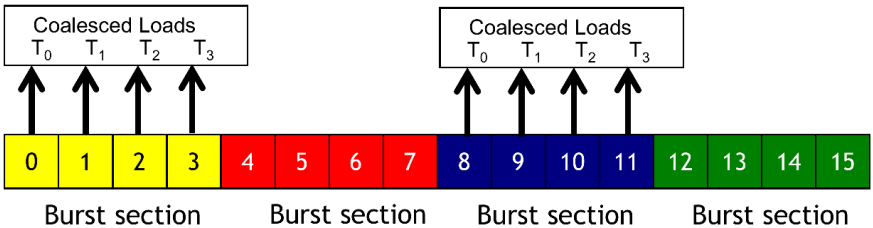


- Digression: DRAM organization
  - Each address space is partitioned into burst sections
  - Whenever a location is accessed, all other locations in the same section are also delivered to the processor



Memory Coalescing (2/2)

- Assume all threads of a warp execute a load instruction where all accessed locations fall into the same burst section ⇒ Only one DRAM request will be made, i.e. the access is fully **coalesced**



Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. **Multi-Device Communication**
7. Final

GPU-Aware MPI

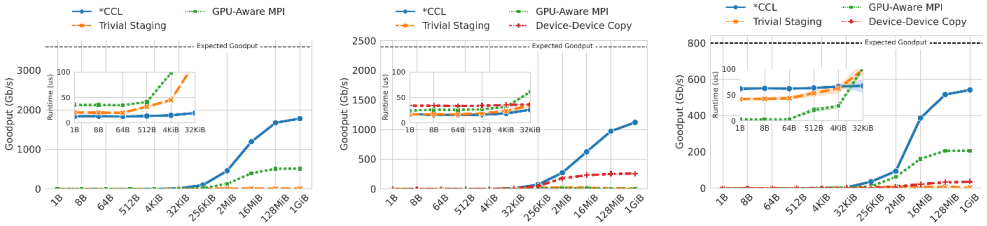


- **Challenge**: By default, MPI can only communicate data in the host memory
  - Before send (after receive) data needs to be copied from device to host (host to device)
- **Idea**: Allow network cards to transfer data from and to device
  - Benefit: Data is directly moved from device memory to network (and vice versa) without using host memory or host cores
  - Role of the host: Setup and guide the communication

GPU-to-GPU Communication

- **Challenge**: Collective communication operations involving all GPUs within a node
  - Should only use fast links connecting the GPUs
  - Possible difficulty: Complex topology
- **Solution**: Specialised libraries
  - NVIDIA Collective Communications Library (NCCL)
  - AMD ROCm Collective Communication Library (RCCL)

Example: Intra-node all-reduce performance [D. de Sensi et al.; 2024]



(a) Alps (b) Leonardo (c) LUMI

Content



1. I/O Performance Engineering
2. GPU Hardware Architectures
3. GPU Programming Models
4. Device Utilization and Data Transfer
5. Device Kernel Optimization
6. Multi-Device Communication
7. **Final**

Summary Lecture #16



- I/O performance optimization techniques
  - Use container file formats
  - Improve I/O patterns
  - Use OS I/O buffers or I/O cache solutions
  - Do parallel I/O cooperatively
- Overview of different Hardware architectures and programming models
- Performance challenges
  - Device utilization
    - Host-device data transfer
  - Device kernel optimization
    - Optimizing concurrency: high occupancy, avoid control divergence
    - Efficient use of memory hierarchy: data reuse, memory coalesing
  - Efficient communication: GPU-aware MPI, GPU-GPU communication

Questions?



[Wiki Media]

These slides contain content and figures from

- Wen-mei Hwu et al., “GPU Teaching Kit”, 2020